

# SQL Cheat Sheet

A practical quick reference for beginners & intermediate users  
Ordered from the most commonly used statements to the least

MySQL

PostgreSQL

SQL Server

## EVERY EXAMPLE IN THIS GUIDE USES THIS TABLE

```
-- the table used by every example in this guide
CREATE TABLE employees (
  Employee_ID VARCHAR(10) PRIMARY KEY,
  Full_Name   VARCHAR(60),
  Department  VARCHAR(30),
  Position    VARCHAR(40),
  Hired_Date  DATE,
  Salary      DECIMAL(10,2),
  Age         INT,
  ...         -- 14 columns, 247 rows
);
```

## FOUR IDEAS THAT EXPLAIN MOST OF SQL

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| SELECT ... FROM | Every query starts here: what columns, from what table.     |
| WHERE vs HAVING | WHERE filters rows; HAVING filters groups after GROUP BY.   |
| JOIN            | Glues two tables together on a column they share.           |
| Dialect         | The same idea, spelled differently in each database engine. |

## HOW THE SAMPLE OUTPUTS WERE PRODUCED

|            |                                               |                            |
|------------|-----------------------------------------------|----------------------------|
| MySQL      | MariaDB 11.8, running live in the sandbox     | output executed            |
| PostgreSQL | PostgreSQL 17.10, running live in the sandbox | output executed            |
| SQL Server | cannot run on this machine                    | syntax from Microsoft docs |

104 examples

33 show dialect differences

14 topics + comparison tables

Companion to the Pandas, NumPy, Scikit-Learn & Matplotlib cheat sheets — same HR data throughout.

# Table of Contents

|           |                                                                                   |          |           |           |
|-----------|-----------------------------------------------------------------------------------|----------|-----------|-----------|
| <b>01</b> | <b>Selecting Data</b><br>Read rows out of a table.                                | 8 items  | 2 differ  | <b>3</b>  |
| <b>02</b> | <b>Filtering with WHERE</b><br>Keep only the rows you want.                       | 10 items | 1 differ  | <b>5</b>  |
| <b>03</b> | <b>Sorting &amp; Limiting</b><br>Put rows in order.                               | 6 items  | 2 differ  | <b>8</b>  |
| <b>04</b> | <b>Aggregate Functions</b><br>Summarise many rows into one.                       | 8 items  | 1 differ  | <b>10</b> |
| <b>05</b> | <b>GROUP BY &amp; HAVING</b><br>Summarise per category.                           | 7 items  | 1 differ  | <b>12</b> |
| <b>06</b> | <b>Joining Tables</b><br>Combine rows from two tables.                            | 8 items  | 1 differ  | <b>14</b> |
| <b>07</b> | <b>Working with Dates</b><br>The biggest dialect difference.                      | 11 items | 11 differ | <b>16</b> |
| <b>08</b> | <b>Text &amp; String Functions</b><br>Clean and combine text.                     | 9 items  | 5 differ  | <b>20</b> |
| <b>09</b> | <b>CASE &amp; Conditional Logic</b><br>Add if-then rules to a query.              | 5 items  | 1 differ  | <b>23</b> |
| <b>10</b> | <b>Subqueries &amp; CTEs</b><br>Use one query inside another.                     | 6 items  |           | <b>25</b> |
| <b>11</b> | <b>Window Functions</b><br>Rank rows without collapsing them.                     | 7 items  |           | <b>27</b> |
| <b>12</b> | <b>Insert, Update &amp; Delete</b><br>Change the data in a table.                 | 6 items  | 1 differ  | <b>29</b> |
| <b>13</b> | <b>Creating &amp; Altering Tables</b><br>Build the structure that holds data.     | 7 items  | 4 differ  | <b>31</b> |
| <b>14</b> | <b>Views &amp; Handy Extras</b><br>Save queries and useful odds and ends.         | 6 items  | 3 differ  | <b>33</b> |
| <b>15</b> | <b>Dialect Comparison Tables</b><br>Every difference in this guide, side by side. | 38 items |           | <b>35</b> |

# 1. Selecting Data

8 functions

Read rows out of a table.

**1.1** `SELECT * FROM employees;` # Gets every column and row

## SAMPLE OUTPUT

| Employee_ID | Full_Name       | Gender | Department  | Position             | Hired_Date |
|-------------|-----------------|--------|-------------|----------------------|------------|
| PH-1004     | Ramon Ilagan    | Male   | IT          | Security Analyst     | 2020-08-11 |
| PH-1006     | Noel Pascual    | Male   | Finance     | Payroll Officer      | 2025-05-24 |
| PH-1007     | Rodel Uy        | Male   | IT          | System Administrator | 2023-06-20 |
| PH-1009     | Vicente Valdez  | Male   | Operations  | Operations Associate | 2021-06-25 |
| PH-1011     | Oscar Zamora    | Male   | Engineering | Software Engineer    | 2019-05-22 |
| PH-1019     | Oscar Fernandez | Male   | Finance     | Payroll Officer      | 2025-03-15 |

**1.2** # Gets only the columns you need  
`SELECT Full_Name, Department, Salary`  
`FROM employees;`

## SAMPLE OUTPUT

| Full_Name       | Department  | Salary    |
|-----------------|-------------|-----------|
| Alvin Aglipay   | Engineering | 69500.00  |
| Alvin Alcantara | Engineering | 91500.00  |
| Alvin Bautista  | Finance     | 135500.00 |
| Alvin Luna      | Engineering | 100000.00 |
| Alvin Ocampo    | Marketing   | NULL      |
| Alvin Ramos     | Engineering | 102000.00 |

**1.3** `SELECT DISTINCT Department FROM employees;` # Lists unique values only

## SAMPLE OUTPUT

| Department      |
|-----------------|
| Engineering     |
| Finance         |
| Human Resources |
| IT              |
| Marketing       |
| Operations      |

**1.4** `SELECT COUNT(*) AS total_staff FROM employees;` # Counts all the rows

## SAMPLE OUTPUT

| total_staff |
|-------------|
| 247         |

**1.5** # Renames columns in the result  
`SELECT Full_Name AS name, Salary AS pay`  
`FROM employees;`

## SAMPLE OUTPUT

| name            | pay       |
|-----------------|-----------|
| Alvin Aglipay   | 69500.00  |
| Alvin Alcantara | 91500.00  |
| Alvin Bautista  | 135500.00 |
| Alvin Luna      | 100000.00 |
| Alvin Ocampo    | NULL      |
| Alvin Ramos     | 102000.00 |

## 1.6 Limits how many rows come back

## MySQL

verified on live server

```
SELECT * FROM employees
LIMIT 5;
```

## PostgreSQL

verified on live server

```
SELECT * FROM employees
LIMIT 5;
```

## SQL Server (T-SQL)

reference only

```
SELECT TOP 5 * FROM employees;
```

## MySQL result

| Employee_ID | Full_Name      | Gender | Department  | Position             | Hired_Date |
|-------------|----------------|--------|-------------|----------------------|------------|
| PH-1004     | Ramon Ilagan   | Male   | IT          | Security Analyst     | 2020-08-11 |
| PH-1006     | Noel Pascual   | Male   | Finance     | Payroll Officer      | 2025-05-24 |
| PH-1007     | Rodel Uy       | Male   | IT          | System Administrator | 2023-06-20 |
| PH-1009     | Vicente Valdez | Male   | Operations  | Operations Associate | 2021-06-25 |
| PH-1011     | Oscar Zamora   | Male   | Engineering | Software Engineer    | 2019-05-22 |

## PostgreSQL result

| employee_id | full_name      | gender | department  | position             | hired_date |
|-------------|----------------|--------|-------------|----------------------|------------|
| PH-1004     | Ramon Ilagan   | Male   | IT          | Security Analyst     | 2020-08-11 |
| PH-1006     | Noel Pascual   | Male   | Finance     | Payroll Officer      | 2025-05-24 |
| PH-1007     | Rodel Uy       | Male   | IT          | System Administrator | 2023-06-20 |
| PH-1009     | Vicente Valdez | Male   | Operations  | Operations Associate | 2021-06-25 |
| PH-1011     | Oscar Zamora   | Male   | Engineering | Software Engineer    | 2019-05-22 |

(5 rows)

**Why they differ:** MySQL/Postgres put LIMIT at the end; SQL Server puts TOP at the start.

## 1.7 Skips rows then takes some

## MySQL

verified on live server

```
SELECT * FROM employees
ORDER BY Salary DESC
LIMIT 5 OFFSET 10;
```

## PostgreSQL

verified on live server

```
SELECT * FROM employees
ORDER BY Salary DESC
LIMIT 5 OFFSET 10;
```

## SQL Server (T-SQL)

reference only

```
SELECT * FROM employees
ORDER BY Salary DESC
OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY;
```

## MySQL result

| Employee_ID | Full_Name         | Gender | Department  | Position            | Hired_Date |
|-------------|-------------------|--------|-------------|---------------------|------------|
| PH-1105     | Dennis Mendoza    | Male   | Engineering | Engineering Manager | 2025-03-15 |
| PH-1641     | Rosario Bonifacio | Female | Engineering | QA Engineer         | 2022-01-11 |
| PH-1051     | Samuel Ocampo     | Male   | Finance     | Finance Manager     | 2024-12-10 |
| PH-1855     | Andrea Mendoza    | Female | Finance     | Finance Manager     | 2023-12-08 |
| PH-1976     | Liza Ocampo       | Female | Finance     | Finance Manager     | 2020-01-03 |

## PostgreSQL result

| employee_id | full_name         | gender | department  | position            | hired_date |
|-------------|-------------------|--------|-------------|---------------------|------------|
| PH-1105     | Dennis Mendoza    | Male   | Engineering | Engineering Manager | 2025-03-15 |
| PH-1641     | Rosario Bonifacio | Female | Engineering | QA Engineer         | 2022-01-11 |
| PH-1051     | Samuel Ocampo     | Male   | Finance     | Finance Manager     | 2024-12-10 |
| PH-1855     | Andrea Mendoza    | Female | Finance     | Finance Manager     | 2023-12-08 |
| PH-1976     | Liza Ocampo       | Female | Finance     | Finance Manager     | 2020-01-03 |

(5 rows)

**Why they differ:** SQL Server needs ORDER BY before OFFSET/FETCH. This also works in Postgres.

## 1.8

```
# Calculates a new column
SELECT Full_Name, Salary * 12 AS annual_pay
FROM employees;
```

## SAMPLE OUTPUT

| Full_Name       | annual_pay |
|-----------------|------------|
| Alvin Aglipay   | 834000.00  |
| Alvin Alcantara | 1098000.00 |
| Alvin Bautista  | 1626000.00 |
| Alvin Luna      | 1200000.00 |
| Alvin Ocampo    | NULL       |
| Alvin Ramos     | 1224000.00 |

## 2. Filtering with WHERE

10 functions

Keep only the rows you want.

**2.1** # Matches one exact value  
 SELECT Full\_Name, Department FROM employees  
 WHERE Department = 'Engineering';

### SAMPLE OUTPUT

| Full_Name        | Department  |
|------------------|-------------|
| Alvin Aglipay    | Engineering |
| Alvin Alcantara  | Engineering |
| Alvin Luna       | Engineering |
| Alvin Ramos      | Engineering |
| Amalia Alonzo    | Engineering |
| Amalia Fernandez | Engineering |

**2.2** # Compares a number  
 SELECT Full\_Name, Salary FROM employees  
 WHERE Salary > 100000;

### SAMPLE OUTPUT

| Full_Name       | Salary    |
|-----------------|-----------|
| Alvin Bautista  | 135500.00 |
| Alvin Ramos     | 102000.00 |
| Amalia Alonzo   | 106500.00 |
| Amalia Bautista | 110000.00 |
| Andrea Espiritu | 125000.00 |
| Andrea Mendoza  | 127000.00 |

**2.3** # Both conditions must be true  
 SELECT Full\_Name, Salary, Age FROM employees  
 WHERE Salary > 80000 AND Age < 30;

### SAMPLE OUTPUT

| Full_Name         | Salary    | Age |
|-------------------|-----------|-----|
| Amalia Alonzo     | 106500.00 | 24  |
| Dennis Salvador   | 105000.00 | 27  |
| Fernando Ilagan   | 93500.00  | 22  |
| Hector Zamora     | 91000.00  | 25  |
| Katrina Salvador  | 148000.00 | 23  |
| Lorenzo Hernandez | 106500.00 | 26  |

**2.4** # Either condition can be true  
 SELECT Full\_Name, Age FROM employees  
 WHERE Age < 25 OR Age > 58;

### SAMPLE OUTPUT

| Full_Name          | Age |
|--------------------|-----|
| Amalia Alonzo      | 24  |
| Amalia Fernandez   | 59  |
| Angelica Aglipay   | 59  |
| Bernadette Mendoza | 22  |
| Carlo Bautista     | 59  |
| Elmer Fernandez    | 23  |

**2.5** # Matches any value in a list  
 SELECT Full\_Name, Office\_Location FROM employees  
 WHERE Office\_Location IN ('Cebu City', 'Clark');

**SAMPLE OUTPUT**

| Full_Name        | Office_Location |
|------------------|-----------------|
| Alvin Aglipay    | Clark           |
| Alvin Ocampo     | Clark           |
| Amalia Bautista  | Cebu City       |
| Andrea Bonifacio | Cebu City       |
| Andrea Domingo   | Cebu City       |
| Andrea Lacsamana | Cebu City       |

**2.6** # Matches a range, ends included  
 SELECT Full\_Name, Age FROM employees  
 WHERE Age BETWEEN 30 AND 35;

**SAMPLE OUTPUT**

| Full_Name             | Age |
|-----------------------|-----|
| Andrea Espiritu       | 33  |
| Angelica Zamora       | 35  |
| Bea Cruz              | 33  |
| Bea Padilla           | 33  |
| Benjamin Zamora       | 34  |
| Bernadette Gatchalian | 35  |

**2.7** # Finds text containing a word  
 SELECT Full\_Name, Position FROM employees  
 WHERE Position LIKE '%Manager%';

**SAMPLE OUTPUT**

| Full_Name        | Position           |
|------------------|--------------------|
| Alvin Bautista   | Finance Manager    |
| Andrea Espiritu  | Sales Manager      |
| Andrea Lacsamana | Marketing Manager  |
| Andrea Mendoza   | Finance Manager    |
| Bea Hernandez    | Sales Manager      |
| Bea Padilla      | Operations Manager |

**2.8** # Finds missing values  
 SELECT Employee\_ID, Full\_Name FROM employees  
 WHERE Salary IS NULL;

**SAMPLE OUTPUT**

| Employee_ID | Full_Name          |
|-------------|--------------------|
| PH-1106     | Marlon Uy          |
| PH-1480     | Erlinda Cabrera    |
| PH-1542     | Zenaida Espiritu   |
| PH-1623     | Kristine Cruz      |
| PH-1643     | Cristina Alcantara |
| PH-1654     | Maricel Cabrera    |

**2.9** # Excludes a value  
 SELECT Full\_Name FROM employees  
 WHERE Department <> 'Sales';

**SAMPLE OUTPUT**

```
+-----+
| Full_Name |
+-----+
| Alvin Aglipay |
| Alvin Alcantara |
| Alvin Bautista |
| Alvin Luna |
| Alvin Ocampo |
| Alvin Ramos |
+-----+
```

**2.10 Ignores upper and lower case****MySQL**

verified on live server

```
SELECT Full_Name FROM employees
WHERE Full_Name LIKE 'alvin%';
```

**PostgreSQL**

verified on live server

```
SELECT Full_Name FROM employees
WHERE Full_Name ILIKE 'alvin%';
```

**SQL Server (T-SQL)**

reference only

```
SELECT Full_Name FROM employees
WHERE Full_Name LIKE 'alvin%';
```

**MySQL result**

```
+-----+
| Full_Name |
+-----+
| Alvin Aglipay |
| Alvin Alcantara |
| Alvin Bautista |
| Alvin Luna |
| Alvin Ocampo |
| Alvin Ramos |
+-----+
```

**PostgreSQL result**

```
full_name
-----
Alvin Aglipay
Alvin Alcantara
Alvin Bautista
Alvin Luna
Alvin Ocampo
Alvin Ramos
(6 rows)
```

**Why they differ:** MySQL and SQL Server ignore case by default. Postgres is case-sensitive, so use ILIKE.

## 3. Sorting & Limiting

6 functions

Put rows in order.

**3.1** # Highest values first  
 SELECT Full\_Name, Salary FROM employees  
 ORDER BY Salary DESC;

### SAMPLE OUTPUT

| Full_Name         | Salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |
| Divina Soriano    | 150000.00 |
| Katrina Salvador  | 148000.00 |
| Oscar Cruz        | 142000.00 |

**3.2** # Lowest values first  
 SELECT Full\_Name, Salary FROM employees  
 ORDER BY Salary ASC;

### SAMPLE OUTPUT

| Full_Name       | Salary   |
|-----------------|----------|
| Bea Cruz        | 33000.00 |
| Bea Quijano     | 34000.00 |
| Divina Espiritu | 34000.00 |
| Nestor Santos   | 37000.00 |
| Zenaida Valdez  | 38000.00 |
| Josefina Guzman | 38500.00 |

**3.3** # Sorts by two columns  
 SELECT Full\_Name, Department, Salary FROM employees  
 WHERE Salary IS NOT NULL  
 ORDER BY Department, Salary DESC, Full\_Name;

### SAMPLE OUTPUT

| Full_Name         | Department  | Salary    |
|-------------------|-------------|-----------|
| Wilfredo Mendoza  | Engineering | 184000.00 |
| Teresita Espiritu | Engineering | 159000.00 |
| Divina Soriano    | Engineering | 150000.00 |
| Dennis Mendoza    | Engineering | 128500.00 |
| Rosario Bonifacio | Engineering | 128500.00 |
| Emilio Ocampo     | Engineering | 125000.00 |

### 3.4 Gets the top earners

#### MySQL

verified on live server

```
SELECT Full_Name, Salary FROM employees
ORDER BY Salary DESC
LIMIT 3;
```

#### PostgreSQL

verified on live server

```
SELECT Full_Name, Salary FROM employees
ORDER BY Salary DESC
LIMIT 3;
```

#### SQL Server (T-SQL)

reference only

```
SELECT TOP 3 Full_Name, Salary FROM employees
ORDER BY Salary DESC;
```

#### MySQL result

| Full_Name         | Salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |

#### PostgreSQL result

| full_name         | salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |

(3 rows)

**Why they differ:** Same TOP vs LIMIT split as before.



**3.5 Puts empty values last****MySQL**

verified on live server

```
SELECT Full_Name, Salary FROM employees
ORDER BY Salary IS NULL, Salary DESC;
```

**PostgreSQL**

verified on live server

```
SELECT Full_Name, Salary FROM employees
ORDER BY Salary DESC NULLS LAST;
```

**SQL Server (T-SQL)**

reference only

```
SELECT Full_Name, Salary FROM employees
ORDER BY CASE WHEN Salary IS NULL THEN 1 ELSE 0 END, Salary DESC;
```

**MySQL result**

| Full_Name         | Salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |
| Divina Soriano    | 150000.00 |
| Katrina Salvador  | 148000.00 |
| Oscar Cruz        | 142000.00 |

**PostgreSQL result**

| full_name         | salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |
| Divina Soriano    | 150000.00 |
| Katrina Salvador  | 148000.00 |
| Oscar Cruz        | 142000.00 |

(6 rows)

**Why they differ:** Only Postgres has NULLS LAST. Others need a trick to push NULLs down.

**3.6**

```
# Sorts by a calculated column
SELECT Department, COUNT(*) AS staff
FROM employees
GROUP BY Department
ORDER BY staff DESC;
```

**SAMPLE OUTPUT**

| Department      | staff |
|-----------------|-------|
| Engineering     | 58    |
| IT              | 40    |
| Finance         | 38    |
| Operations      | 31    |
| Sales           | 31    |
| Human Resources | 26    |

## 4. Aggregate Functions

8 functions

Summarise many rows into one.

**4.1** `SELECT COUNT(*) FROM employees;` # Counts every row

SAMPLE OUTPUT

```
+-----+
| COUNT(*) |
+-----+
|      247 |
+-----+
```

**4.2** `SELECT COUNT(Salary) FROM employees;` # Counts non-empty values only

SAMPLE OUTPUT

```
+-----+
| COUNT(Salary) |
+-----+
|          235 |
+-----+
```

**4.3** `SELECT ROUND(AVG(Salary), 2) AS avg_salary FROM employees;` # Calculates the average

SAMPLE OUTPUT

```
+-----+
| avg_salary |
+-----+
|  83014.89 |
+-----+
```

**4.4** `SELECT SUM(Salary) AS payroll FROM employees;` # Adds up all values

SAMPLE OUTPUT

```
+-----+
| payroll |
+-----+
| 19508500.00 |
+-----+
```

**4.5** # Smallest and largest values  
`SELECT MIN(Salary) AS lowest, MAX(Salary) AS highest`  
`FROM employees;`

SAMPLE OUTPUT

```
+-----+-----+
| lowest | highest |
+-----+-----+
| 33000.00 | 184000.00 |
+-----+-----+
```

**4.6** `SELECT COUNT(DISTINCT Department) AS depts FROM employees;` # Counts unique values

SAMPLE OUTPUT

```
+-----+
| depts |
+-----+
|      7 |
+-----+
```

**4.7 Finds the middle value**

**MySQL** verified on live server

```
SELECT AVG(Salary) AS median FROM (
  SELECT Salary, ROW_NUMBER() OVER (ORDER BY Salary) AS rn,
    COUNT(*) OVER () AS n
  FROM employees WHERE Salary IS NOT NULL) t
WHERE rn IN (FLOOR((n+1)/2), CEIL((n+1)/2));
```

**PostgreSQL** verified on live server

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
  AS median
FROM employees;
```

**SQL Server (T-SQL)**reference only

```
SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
  OVER () AS median
FROM employees;
```

**MySQL result**

```
+-----+
| median |
+-----+
| 80500.000000 |
+-----+
```

**PostgreSQL result**

```
median
-----
80500
(1 row)
```

**Why they differ:** Postgres PERCENTILE\_CONT is exact. The MySQL workaround returns DECIMAL(x.000000).

**4.8**

```
# Several summaries at once
SELECT ROUND(AVG(Age), 1) AS avg_age,
  ROUND(AVG(Performance_Rating), 2) AS avg_rating
FROM employees;
```

**SAMPLE OUTPUT**

```
+-----+-----+
| avg_age | avg_rating |
+-----+-----+
| 40.9 | 3.52 |
+-----+-----+
```

## 5. GROUP BY & HAVING

7 functions

Summarise per category.

**5.1** # Counts rows in each group  
 SELECT Department, COUNT(\*) AS staff  
 FROM employees  
 GROUP BY Department;

### SAMPLE OUTPUT

| Department      | staff |
|-----------------|-------|
| Engineering     | 58    |
| Finance         | 38    |
| Human Resources | 26    |
| IT              | 40    |
| Marketing       | 23    |
| Operations      | 31    |

**5.2** # Average for each group  
 SELECT Department, ROUND(AVG(Salary), 2) AS avg\_pay  
 FROM employees  
 GROUP BY Department  
 ORDER BY avg\_pay DESC;

### SAMPLE OUTPUT

| Department      | avg_pay  |
|-----------------|----------|
| Engineering     | 98419.64 |
| Finance         | 88986.11 |
| IT              | 79657.89 |
| Human Resources | 76980.77 |
| Sales           | 76383.33 |
| Marketing       | 73886.36 |

**5.3** # Groups by two columns  
 SELECT Department, Gender, COUNT(\*) AS staff  
 FROM employees  
 GROUP BY Department, Gender  
 ORDER BY Department, Gender;

### SAMPLE OUTPUT

| Department      | Gender | staff |
|-----------------|--------|-------|
| Engineering     | Female | 33    |
| Engineering     | Male   | 25    |
| Finance         | Female | 20    |
| Finance         | Male   | 18    |
| Human Resources | Female | 13    |
| Human Resources | Male   | 13    |

**5.4** # Filters the groups, not the rows  
 SELECT Department, COUNT(\*) AS staff  
 FROM employees  
 GROUP BY Department  
 HAVING COUNT(\*) > 30;

### SAMPLE OUTPUT

| Department  | staff |
|-------------|-------|
| Engineering | 58    |
| Finance     | 38    |
| IT          | 40    |
| Operations  | 31    |
| Sales       | 31    |

**5.5**

```
# Several summaries per group
SELECT Employment_Type, ROUND(AVG(Salary), 2) AS avg_pay,
       COUNT(*) AS staff
FROM employees
GROUP BY Employment_Type;
```

**SAMPLE OUTPUT**

```
+-----+-----+-----+
| Employment_Type | avg_pay | staff |
+-----+-----+-----+
| Contractor      | 78068.97 | 34    |
| Full-Time       | 83235.47 | 179   |
| Part-Time       | 86117.65 | 34    |
+-----+-----+-----+
```

**5.6**

```
# WHERE filters rows, HAVING filters groups
SELECT Department, COUNT(*) AS staff
FROM employees
WHERE Employment_Type = 'Full-Time'
GROUP BY Department
HAVING COUNT(*) >= 20
ORDER BY staff DESC;
```

**SAMPLE OUTPUT**

```
+-----+-----+
| Department | staff |
+-----+-----+
| Engineering | 42    |
| IT          | 32    |
| Finance     | 26    |
| Human Resources | 23    |
| Sales       | 23    |
+-----+-----+
```

**5.7 Adds a grand total row**

**MySQL** verified on live server

```
SELECT Department, COUNT(*) AS staff
FROM employees
GROUP BY Department WITH ROLLUP;
```

**PostgreSQL** verified on live server

```
SELECT Department, COUNT(*) AS staff
FROM employees
GROUP BY ROLLUP(Department);
```

**SQL Server (T-SQL)**reference only

```
SELECT Department, COUNT(*) AS staff
FROM employees
GROUP BY ROLLUP(Department);
```

**MySQL result**

```
+-----+-----+
| Department | staff |
+-----+-----+
| Engineering | 58    |
| Finance     | 38    |
| Human Resources | 26    |
| IT          | 40    |
| Marketing   | 23    |
| Operations  | 31    |
+-----+-----+
```

**PostgreSQL result**

```
department | staff
-----+-----
Engineering | 58
Finance     | 38
Human Resources | 26
IT          | 40
Marketing   | 23
Operations  | 31
(6 rows)
```

**Why they differ:** MySQL writes WITH ROLLUP after the columns; the others use ROLLUP(...).

## 6. Joining Tables

8 functions

Combine rows from two tables.

**6.1** # Keeps rows found in both tables  
 SELECT e.Full\_Name, e.Department, d.Head  
 FROM employees e  
 JOIN departments d ON e.Department = d.Department;

SAMPLE OUTPUT

| Full_Name       | Department  | Head             |
|-----------------|-------------|------------------|
| Alvin Aglipay   | Engineering | Ramon Villanueva |
| Alvin Alcantara | Engineering | Ramon Villanueva |
| Alvin Bautista  | Finance     | Teresita Ocampo  |
| Alvin Luna      | Engineering | Ramon Villanueva |
| Alvin Ocampo    | Marketing   | Michelle Domingo |
| Alvin Ramos     | Engineering | Ramon Villanueva |

**6.2** # Keeps all rows from the left table  
 SELECT e.Full\_Name, d.Head, d.Floor  
 FROM employees e  
 LEFT JOIN departments d ON e.Department = d.Department;

SAMPLE OUTPUT

| Full_Name       | Head             | Floor |
|-----------------|------------------|-------|
| Alvin Aglipay   | Ramon Villanueva | 5     |
| Alvin Alcantara | Ramon Villanueva | 5     |
| Alvin Bautista  | Teresita Ocampo  | 3     |
| Alvin Luna      | Ramon Villanueva | 5     |
| Alvin Ocampo    | Michelle Domingo | 6     |
| Alvin Ramos     | Ramon Villanueva | 5     |

**6.3** # Finds groups with no matches  
 SELECT d.Department, COUNT(e.Employee\_ID) AS staff  
 FROM departments d  
 LEFT JOIN employees e ON d.Department = e.Department  
 GROUP BY d.Department  
 ORDER BY staff;

SAMPLE OUTPUT

| Department      | staff |
|-----------------|-------|
| Logistics       | 0     |
| Marketing       | 23    |
| Human Resources | 26    |
| Sales           | 31    |
| Finance         | 38    |
| IT              | 40    |

**6.4** # Rows in one table but not the other  
 SELECT d.Department, d.Head  
 FROM departments d  
 WHERE d.Department NOT IN (SELECT DISTINCT Department FROM employees);

SAMPLE OUTPUT

| Department | Head       |
|------------|------------|
| Logistics  | Unassigned |

**6.5 Keeps unmatched rows from both sides**

**MySQL** verified on live server

```
SELECT e.Full_Name, d.Department FROM employees e
LEFT JOIN departments d ON e.Department = d.Department
UNION
SELECT e.Full_Name, d.Department FROM employees e
RIGHT JOIN departments d ON e.Department = d.Department
ORDER BY 2, 1;
```

**PostgreSQL** verified on live server

```
SELECT e.Full_Name, d.Department
FROM employees e
FULL OUTER JOIN departments d ON e.Department = d.Department
ORDER BY 2, 1;
```

**SQL Server (T-SQL)**reference only

```
SELECT e.Full_Name, d.Department
FROM employees e
FULL OUTER JOIN departments d ON e.Department = d.Department;
```

**MySQL result**

| Full_Name        | Department |
|------------------|------------|
| Andrea Bonifacio | NULL       |
| Bea Cruz         | NULL       |
| Bea Padilla      | NULL       |
| Bea Quijano      | NULL       |
| Carlo Lacsamana  | NULL       |
| Cristina Guzman  | NULL       |

**PostgreSQL result**

| full_name        | department  |
|------------------|-------------|
| Alvin Aglipay    | Engineering |
| Alvin Alcantara  | Engineering |
| Alvin Luna       | Engineering |
| Alvin Ramos      | Engineering |
| Amalia Alonzo    | Engineering |
| Amalia Fernandez | Engineering |

(6 rows)

**Why they differ:** MySQL has NO FULL OUTER JOIN - fake it with UNION. Note MySQL sorts NULLs first.

**6.6**

```
# Joins a table to itself
SELECT a.Full_Name, b.Full_Name AS colleague
FROM employees a
JOIN employees b
  ON a.Department = b.Department AND a.Employee_ID < b.Employee_ID
ORDER BY a.Full_Name, b.Full_Name;
```

**SAMPLE OUTPUT**

| Full_Name     | colleague          |
|---------------|--------------------|
| Alvin Aglipay | Alvin Luna         |
| Alvin Aglipay | Alvin Ramos        |
| Alvin Aglipay | Amalia Fernandez   |
| Alvin Aglipay | Bea Soriano        |
| Alvin Aglipay | Bernadette Cabrera |
| Alvin Aglipay | Bernadette Guzman  |

**6.7**

```
# Stacks results and removes repeats
SELECT Department FROM employees
UNION
SELECT Department FROM departments
ORDER BY Department;
```

**SAMPLE OUTPUT**

| Department      |
|-----------------|
| Engineering     |
| Finance         |
| Human Resources |
| IT              |
| Logistics       |
| Marketing       |

**6.8**

```
# Stacks results and keeps repeats
SELECT Department FROM employees
UNION ALL
SELECT Department FROM departments;
```

**SAMPLE OUTPUT**

| rows_returned |
|---------------|
| 254           |

## 7. Working with Dates

11 functions

The biggest dialect difference.

### 7.1 Gets today's date

**MySQL** verified on live server

```
SELECT CURDATE() AS today;
```

**PostgreSQL** verified on live server

```
SELECT CURRENT_DATE AS today;
```

**SQL Server (T-SQL)** reference only

```
SELECT CAST(GETDATE() AS DATE) AS today;
```

#### MySQL result

```
+-----+
| today |
+-----+
| 2026-07-26 |
+-----+
```

#### PostgreSQL result

```
today
-----
2026-07-26
(1 row)
```

**Why they differ:** Three different function names for the same thing. CURRENT\_DATE works on MySQL and Postgres.

### 7.2 Gets the current date and time

**MySQL** verified on live server

```
SELECT NOW() AS right_now;
```

**PostgreSQL** verified on live server

```
SELECT NOW() AS right_now;
```

**SQL Server (T-SQL)** reference only

```
SELECT GETDATE() AS right_now;
```

#### MySQL result

```
+-----+
| right_now |
+-----+
| 2026-07-26 16:57:04 |
+-----+
```

#### PostgreSQL result

```
right_now
-----
2026-07-26 16:57:04.96574+00
(1 row)
```

**Why they differ:** SQL Server uses GETDATE() or SYSDATETIME(); it has no NOW().

### 7.3 Pulls the year out of a date

**MySQL** verified on live server

```
SELECT Full_Name, YEAR(Hired_Date) AS hire_year
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT Full_Name, EXTRACT(YEAR FROM Hired_Date) AS hire_year
FROM employees;
```

**SQL Server (T-SQL)** reference only

```
SELECT Full_Name, YEAR(Hired_Date) AS hire_year
FROM employees;
```

#### MySQL result

```
+-----+-----+
| Full_Name | hire_year |
+-----+-----+
| Alvin Aglipay | 2022 |
| Alvin Alcantara | 2022 |
| Alvin Bautista | 2018 |
| Alvin Luna | 2026 |
| Alvin Ocampo | 2016 |
| Alvin Ramos | 2022 |
+-----+-----+
```

#### PostgreSQL result

```
full_name | hire_year
-----+-----
Alvin Aglipay | 2022
Alvin Alcantara | 2022
Alvin Bautista | 2018
Alvin Luna | 2026
Alvin Ocampo | 2016
Alvin Ramos | 2022
(6 rows)
```

**Why they differ:** MySQL and SQL Server have YEAR(). Postgres uses EXTRACT (which MySQL also accepts).

### 7.4 Turns text into a real date

**MySQL** verified on live server

```
SELECT STR_TO_DATE('4-Dec-2022', '%d-%b-%Y') AS hired;
```

**PostgreSQL** verified on live server

```
SELECT TO_DATE('4-Dec-2022', 'DD-Mon-YYYY') AS hired;
```

**SQL Server (T-SQL)** reference only

```
SELECT CONVERT(DATE, '4-Dec-2022', 106) AS hired;
```

#### MySQL result

```
+-----+
| hired |
+-----+
| 2022-12-04 |
+-----+
```

#### PostgreSQL result

```
hired
-----
2022-12-04
(1 row)
```

**Why they differ:** THE classic difference: STR\_TO\_DATE vs TO\_DATE vs CONVERT with a style number.



**7.5 Turns a date into formatted text**

**MySQL** verified on live server

```
SELECT DATE_FORMAT(Hired_Date, '%d/%m/%Y') AS hired
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT TO_CHAR(Hired_Date, 'DD/MM/YYYY') AS hired
FROM employees;
```

**SQL Server (T-SQL)** reference only

```
SELECT FORMAT(Hired_Date, 'dd/MM/yyyy') AS hired
FROM employees;
```

**MySQL result**

```
+-----+
| hired |
+-----+
| 01/02/2024 |
| 01/03/2016 |
| 01/03/2016 |
| 01/04/2019 |
| 01/05/2015 |
| 01/06/2017 |
+-----+
```

**PostgreSQL result**

```
hired
-----
01/02/2024
01/03/2016
01/03/2016
01/04/2019
01/05/2015
01/06/2017
(6 rows)
```

**Why they differ:** DATE\_FORMAT vs TO\_CHAR vs FORMAT - and the pattern letters differ too.

**7.6 Adds days to a date**

**MySQL** verified on live server

```
SELECT DATE_ADD(Hired_Date, INTERVAL 30 DAY) AS review_due
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT Hired_Date + INTERVAL '30 days' AS review_due
FROM employees;
```

**SQL Server (T-SQL)** reference only

```
SELECT DATEADD(DAY, 30, Hired_Date) AS review_due
FROM employees;
```

**MySQL result**

```
+-----+
| review_due |
+-----+
| 2015-02-04 |
| 2015-04-24 |
| 2015-05-14 |
| 2015-05-14 |
| 2015-05-31 |
| 2015-06-03 |
+-----+
```

**PostgreSQL result**

```
review_due
-----
2015-02-04 00:00:00
2015-04-24 00:00:00
2015-05-14 00:00:00
2015-05-14 00:00:00
2015-05-31 00:00:00
2015-06-03 00:00:00
(6 rows)
```

**Why they differ:** Postgres returns a TIMESTAMP here - add ::DATE if you want a plain date back.

**7.7 Counts days between two dates**

**MySQL** verified on live server

```
SELECT DATEDIFF(CURDATE(), Hired_Date) AS days_served
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT CURRENT_DATE - Hired_Date AS days_served
FROM employees;
```

**SQL Server (T-SQL)** reference only

```
SELECT DATEDIFF(DAY, Hired_Date, GETDATE()) AS days_served
FROM employees;
```

**MySQL result**

```
+-----+
| days_served |
+-----+
| 121 |
| 133 |
| 143 |
| 149 |
| 171 |
| 205 |
+-----+
```

**PostgreSQL result**

```
days_served
-----
121
133
143
149
171
205
(6 rows)
```

**Why they differ:** Careful: MySQL DATEDIFF takes 2 args, SQL Server takes 3 (unit first).

## 7.8 Calculates whole years of service

**MySQL** verified on live server

```
SELECT Full_Name,
       TIMESTAMPDIFF(YEAR, Hired_Date, CURDATE()) AS yrs
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT Full_Name,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, Hired_Date)) AS yrs
FROM employees;
```

**SQL Server (T-SQL)**reference only

```
SELECT Full_Name,
       DATEDIFF(YEAR, Hired_Date, GETDATE()) AS yrs
FROM employees;
```

**MySQL result**

| Full_Name       | yrs |
|-----------------|-----|
| Alvin Aglipay   | 3   |
| Alvin Alcantara | 4   |
| Alvin Bautista  | 7   |
| Alvin Luna      | 0   |
| Alvin Ocampo    | 9   |
| Alvin Ramos     | 4   |

**PostgreSQL result**

| full_name       | yrs |
|-----------------|-----|
| Alvin Aglipay   | 3   |
| Alvin Alcantara | 4   |
| Alvin Bautista  | 7   |
| Alvin Luna      | 0   |
| Alvin Ocampo    | 9   |
| Alvin Ramos     | 4   |

(6 rows)

**Why they differ:** SQL Server DATEDIFF(YEAR..) counts calendar boundaries, so it can be 1 too high.

## 7.9 Filters rows by year

**MySQL** verified on live server

```
SELECT Full_Name, Hired_Date FROM employees
WHERE YEAR(Hired_Date) = 2022;
```

**PostgreSQL** verified on live server

```
SELECT Full_Name, Hired_Date FROM employees
WHERE Hired_Date >= DATE '2022-01-01'
AND Hired_Date < DATE '2023-01-01';
```

**SQL Server (T-SQL)**reference only

```
SELECT Full_Name, Hired_Date FROM employees
WHERE Hired_Date >= '20220101' AND Hired_Date < '20230101';
```

**MySQL result**

| Full_Name       | Hired_Date |
|-----------------|------------|
| Alvin Aglipay   | 2022-12-04 |
| Alvin Alcantara | 2022-01-31 |
| Alvin Ramos     | 2022-03-30 |
| Amalia Ocampo   | 2022-10-04 |
| Bea Hernandez   | 2022-02-18 |
| Benjamin Aquino | 2022-05-11 |

**PostgreSQL result**

| full_name       | hired_date |
|-----------------|------------|
| Alvin Aglipay   | 2022-12-04 |
| Alvin Alcantara | 2022-01-31 |
| Alvin Ramos     | 2022-03-30 |
| Amalia Ocampo   | 2022-10-04 |
| Bea Hernandez   | 2022-02-18 |
| Benjamin Aquino | 2022-05-11 |

(6 rows)

**Why they differ:** A range beats YEAR() everywhere: it can use an index, YEAR() cannot.

## 7.10 Groups hires by month

**MySQL** verified on live server

```
SELECT DATE_FORMAT(Hired_Date, '%Y-%m') AS month,
       COUNT(*) AS hires
FROM employees
GROUP BY month
ORDER BY month
LIMIT 6;
```

**PostgreSQL** verified on live server

```
SELECT TO_CHAR(Hired_Date, 'YYYY-MM') AS month,
       COUNT(*) AS hires
FROM employees
GROUP BY month
ORDER BY month
LIMIT 6;
```

**SQL Server (T-SQL)**reference only

```
SELECT FORMAT(Hired_Date, 'yyyy-MM') AS month,
       COUNT(*) AS hires
FROM employees
GROUP BY FORMAT(Hired_Date, 'yyyy-MM')
ORDER BY month;
```

**MySQL result**

| month   | hires |
|---------|-------|
| 2015-01 | 1     |
| 2015-03 | 1     |
| 2015-04 | 2     |
| 2015-05 | 7     |
| 2015-07 | 1     |
| 2015-08 | 2     |

**PostgreSQL result**

| month   | hires |
|---------|-------|
| 2015-01 | 1     |
| 2015-03 | 1     |
| 2015-04 | 2     |
| 2015-05 | 7     |
| 2015-07 | 1     |
| 2015-08 | 2     |

(6 rows)

**Why they differ:** SQL Server cannot GROUP BY an alias - repeat the whole expression.

7.11 Gets the first day of the month

|                                                                                                                                                                                                          |                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>MySQL</div> <div>verified on live server</div> <div>SELECT DATE_FORMAT(Hired_Date, '%Y-%m-01') AS month_start<br/>FROM employees;</div>                                                             | <div>PostgreSQL</div> <div>verified on live server</div> <div>SELECT DATE_TRUNC('month', Hired_Date)::DATE AS month_start<br/>FROM employees;</div>                  |
| <div>SQL Server (T-SQL)</div> <div>reference only</div> <div>SELECT DATEFROMPARTS(YEAR(Hired_Date), MONTH(Hired_Date), 1)<br/>AS month_start<br/>FROM employees;</div>                                   |                                                                                                                                                                      |
| <div>MySQL result</div> <div>+-----+<br/>  month_start  <br/>+-----+<br/>  2015-01-01  <br/>  2015-03-01  <br/>  2015-04-01  <br/>  2015-04-01  <br/>  2015-05-01  <br/>  2015-05-01  <br/>+-----+</div> | <div>PostgreSQL result</div> <div>month_start<br/>-----<br/>2015-01-01<br/>2015-03-01<br/>2015-04-01<br/>2015-04-01<br/>2015-05-01<br/>2015-05-01<br/>(6 rows)</div> |

**Why they differ:** DATE\_TRUNC is a Postgres speciality and very handy for reports.

## 8. Text & String Functions

9 functions

Clean and combine text.

### 8.1 Joins text together

#### MySQL

verified on live server

```
SELECT CONCAT(Full_Name, ' - ', Department) AS label
FROM employees;
```

#### PostgreSQL

verified on live server

```
SELECT Full_Name || ' - ' || Department AS label
FROM employees;
```

#### SQL Server (T-SQL)

reference only

```
SELECT Full_Name + ' - ' + Department AS label
FROM employees;
```

#### MySQL result

```
+-----+
| label |
+-----+
| Alvin Aglipay - Engineering |
| Alvin Alcantara - Engineering |
| Alvin Bautista - Finance |
| Alvin Luna - Engineering |
| Alvin Ocampo - Marketing |
| Alvin Ramos - Engineering |
+-----+
```

#### PostgreSQL result

```
label
-----
Alvin Aglipay - Engineering
Alvin Alcantara - Engineering
Alvin Bautista - Finance
Alvin Luna - Engineering
Alvin Ocampo - Marketing
Alvin Ramos - Engineering
(6 rows)
```

**Why they differ:** || in Postgres, + in SQL Server, CONCAT() works on all three.

### 8.2 SELECT UPPER(Full\_Name) AS shouty FROM employees; # Converts text to capitals

#### SAMPLE OUTPUT

```
+-----+
| shouty |
+-----+
| ALVIN AGLIPAY |
| ALVIN ALCANTARA |
| ALVIN BAUTISTA |
| ALVIN LUNA |
| ALVIN OCAMPO |
| ALVIN RAMOS |
+-----+
```

### 8.3 # Converts text to lowercase SELECT LOWER(Email) AS email FROM employees WHERE Email IS NOT NULL;

#### SAMPLE OUTPUT

```
+-----+
| email |
+-----+
| alvin.aglipay@apexsolutions.com.ph |
| alvin.alcantara@apexsolutions.com.ph |
| alvin.bautista@apexsolutions.com.ph |
| alvin.luna@apexsolutions.com.ph |
| alvin.ocampo@apexsolutions.com.ph |
| alvin.ramos@apexsolutions.com.ph |
+-----+
```

### 8.4 Counts characters in text

#### MySQL

verified on live server

```
SELECT Full_Name, LENGTH(Full_Name) AS chars
FROM employees;
```

#### PostgreSQL

verified on live server

```
SELECT Full_Name, LENGTH(Full_Name) AS chars
FROM employees;
```

#### SQL Server (T-SQL)

reference only

```
SELECT Full_Name, LEN(Full_Name) AS chars
FROM employees;
```

#### MySQL result

```
+-----+-----+
| Full_Name | chars |
+-----+-----+
| Alvin Aglipay | 13 |
| Alvin Alcantara | 15 |
| Alvin Bautista | 14 |
| Alvin Luna | 10 |
| Alvin Ocampo | 12 |
| Alvin Ramos | 11 |
+-----+-----+
```

#### PostgreSQL result

```
full_name | chars
-----+-----
Alvin Aglipay | 13
Alvin Alcantara | 15
Alvin Bautista | 14
Alvin Luna | 10
Alvin Ocampo | 12
Alvin Ramos | 11
(6 rows)
```

**Why they differ:** SQL Server uses LEN(). Also note MySQL LENGTH() counts bytes, not characters.

**8.5 Takes the first few characters****MySQL**

verified on live server

```
SELECT SUBSTRING(Employee_ID, 4, 4) AS number
FROM employees;
```

**PostgreSQL**

verified on live server

```
SELECT SUBSTRING(Employee_ID, 4, 4) AS number
FROM employees;
```

**SQL Server (T-SQL)**

reference only

```
SELECT SUBSTRING(Employee_ID, 4, 4) AS number
FROM employees;
```

**MySQL result**

```
+-----+
| number |
+-----+
| 1004   |
| 1006   |
| 1007   |
| 1009   |
| 1011   |
| 1019   |
+-----+
```

**PostgreSQL result**

```
number
-----
1004
1006
1007
1009
1011
1019
(6 rows)
```

**Why they differ:** SUBSTRING is the same everywhere. MySQL/SQL Server also allow LEFT() and RIGHT().

**8.6**

```
# Swaps one piece of text for another
SELECT REPLACE(Phone_Number, ' ', '') AS phone
FROM employees;
```

**SAMPLE OUTPUT**

```
+-----+
| phone |
+-----+
| +639103367737 |
| +639106971468 |
| +639117188135 |
| +639118983096 |
| +639119422255 |
| +639119547772 |
+-----+
```

**8.7**

```
SELECT TRIM(Office_Location) AS office FROM employees;    # Removes spaces from both ends
```

**SAMPLE OUTPUT**

```
+-----+
| office |
+-----+
| Cagayan de Oro |
| Cagayan de Oro |
| Cagayan de Oro |
| Cagayan de Oro |
| Cagayan de Oro |
| Cagayan de Oro |
+-----+
```

**8.8 Finds where text appears****MySQL**

verified on live server

```
SELECT LOCATE('@', Email) AS at_pos FROM employees
WHERE Email IS NOT NULL;
```

**PostgreSQL**

verified on live server

```
SELECT POSITION('@' IN Email) AS at_pos FROM employees
WHERE Email IS NOT NULL;
```

**SQL Server (T-SQL)**

reference only

```
SELECT CHARINDEX('@', Email) AS at_pos FROM employees
WHERE Email IS NOT NULL;
```

**MySQL result**

```
+-----+
| at_pos |
+-----+
| 8       |
| 9       |
| 9       |
| 10      |
| 10      |
| 10      |
+-----+
```

**PostgreSQL result**

```
at_pos
-----
8
9
9
10
10
10
(6 rows)
```

**Why they differ:** Three different names: LOCATE, POSITION and CHARINDEX.

**8.9 Replaces NULL with a fallback****MySQL**

verified on live server

```
SELECT Full_Name, IFNULL(Manager, 'No manager') AS mgr
FROM employees;
```

**PostgreSQL**

verified on live server

```
SELECT Full_Name, COALESCE(Manager, 'No manager') AS mgr
FROM employees;
```

**SQL Server (T-SQL)**

reference only

```
SELECT Full_Name, ISNULL(Manager, 'No manager') AS mgr
FROM employees;
```

**MySQL result**

| Full_Name       | mgr              |
|-----------------|------------------|
| Alvin Aglipay   | Ramon Villanueva |
| Alvin Alcantara | Ramon Villanueva |
| Alvin Bautista  | Teresita Ocampo  |
| Alvin Luna      | Ramon Villanueva |
| Alvin Ocampo    | Michelle Domingo |
| Alvin Ramos     | Ramon Villanueva |

**PostgreSQL result**

| full_name       | mgr              |
|-----------------|------------------|
| Alvin Aglipay   | Ramon Villanueva |
| Alvin Alcantara | Ramon Villanueva |
| Alvin Bautista  | Teresita Ocampo  |
| Alvin Luna      | Ramon Villanueva |
| Alvin Ocampo    | Michelle Domingo |
| Alvin Ramos     | Ramon Villanueva |

(6 rows)

**Why they differ:** COALESCE is standard SQL and works on all three - prefer it.

## 9. CASE & Conditional Logic

5 functions

Add if-then rules to a query.

```
9.1 # Labels rows by a rule
SELECT Full_Name, Salary,
       CASE WHEN Salary > 100000 THEN 'High'
            WHEN Salary > 70000 THEN 'Medium'
            ELSE 'Low' END AS band
FROM employees;
```

### SAMPLE OUTPUT

| Full_Name       | Salary    | band   |
|-----------------|-----------|--------|
| Alvin Aglipay   | 69500.00  | Low    |
| Alvin Alcantara | 91500.00  | Medium |
| Alvin Bautista  | 135500.00 | High   |
| Alvin Luna      | 100000.00 | Medium |
| Alvin Ocampo    | NULL      | Low    |
| Alvin Ramos     | 102000.00 | High   |

```
9.2 # Counts groups sideways, like a pivot
SELECT Department,
       SUM(CASE WHEN Gender = 'Female' THEN 1 ELSE 0 END) AS women,
       SUM(CASE WHEN Gender = 'Male' THEN 1 ELSE 0 END) AS men
FROM employees
GROUP BY Department;
```

### SAMPLE OUTPUT

| Department      | women | men |
|-----------------|-------|-----|
| Engineering     | 33    | 25  |
| Finance         | 20    | 18  |
| Human Resources | 13    | 13  |
| IT              | 20    | 20  |
| Marketing       | 12    | 11  |
| Operations      | 15    | 16  |

```
9.3 # Handles missing values in a rule
SELECT Full_Name,
       CASE WHEN Salary IS NULL THEN 'Not recorded'
            ELSE 'Recorded' END AS status
FROM employees;
```

### SAMPLE OUTPUT

| Full_Name       | status       |
|-----------------|--------------|
| Alvin Aglipay   | Recorded     |
| Alvin Alcantara | Recorded     |
| Alvin Bautista  | Recorded     |
| Alvin Luna      | Recorded     |
| Alvin Ocampo    | Not recorded |
| Alvin Ramos     | Recorded     |

## 9.4 Short version of a simple IF

**MySQL** verified on live server

```
SELECT Full_Name, IF(Salary > 90000, 'High', 'Low') AS band
FROM employees;
```

**PostgreSQL** verified on live server

```
SELECT Full_Name,
CASE WHEN Salary > 90000 THEN 'High' ELSE 'Low' END AS band
FROM employees;
```

**SQL Server (T-SQL)**reference only

```
SELECT Full_Name, IIF(Salary > 90000, 'High', 'Low') AS band
FROM employees;
```

**MySQL result**

| Full_Name       | band |
|-----------------|------|
| Alvin Aglipay   | Low  |
| Alvin Alcantara | High |
| Alvin Bautista  | High |
| Alvin Luna      | High |
| Alvin Ocampo    | Low  |
| Alvin Ramos     | High |

**PostgreSQL result**

| full_name       | band |
|-----------------|------|
| Alvin Aglipay   | Low  |
| Alvin Alcantara | High |
| Alvin Bautista  | High |
| Alvin Luna      | High |
| Alvin Ocampo    | Low  |
| Alvin Ramos     | High |

(6 rows)

**Why they differ:** MySQL has IF(), SQL Server has IIF(), Postgres has neither - use CASE.

## 9.5

```
# Averages only the matching rows
SELECT Department,
ROUND(AVG(CASE WHEN Employment_Type = 'Full-Time'
THEN Salary END), 2) AS fulltime_avg
FROM employees
GROUP BY Department;
```

**SAMPLE OUTPUT**

| Department      | fulltime_avg |
|-----------------|--------------|
| Engineering     | 99024.39     |
| Finance         | 90380.00     |
| Human Resources | 74934.78     |
| IT              | 81900.00     |
| Marketing       | 71250.00     |
| Operations      | 67500.00     |



## 10. Subqueries & CTEs

6 functions

Use one query inside another.

**10.1** # Compares rows to a summary  
 SELECT Full\_Name, Salary FROM employees  
 WHERE Salary > (SELECT AVG(Salary) FROM employees)  
 ORDER BY Salary DESC;

### SAMPLE OUTPUT

| Full_Name         | Salary    |
|-------------------|-----------|
| Wilfredo Mendoza  | 184000.00 |
| Fernando Castillo | 160000.00 |
| Teresita Espiritu | 159000.00 |
| Divina Soriano    | 150000.00 |
| Katrina Salvador  | 148000.00 |
| Oscar Cruz        | 142000.00 |

**10.2** # Filters using another query  
 SELECT Full\_Name, Department FROM employees  
 WHERE Department IN (  
 SELECT Department FROM employees  
 GROUP BY Department HAVING COUNT(\*) > 35);

### SAMPLE OUTPUT

| Full_Name       | Department  |
|-----------------|-------------|
| Alvin Aglipay   | Engineering |
| Alvin Alcantara | Engineering |
| Alvin Bautista  | Finance     |
| Alvin Luna      | Engineering |
| Alvin Ramos     | Engineering |
| Alvin Salvador  | IT          |

**10.3** # A query that runs per row  
 SELECT d.Department,  
 (SELECT COUNT(\*) FROM employees e  
 WHERE e.Department = d.Department) AS staff  
 FROM departments d;

### SAMPLE OUTPUT

| Department      | staff |
|-----------------|-------|
| Engineering     | 58    |
| Finance         | 38    |
| Human Resources | 26    |
| IT              | 40    |
| Logistics       | 0     |
| Marketing       | 23    |

**10.4** # Names a query to reuse it

```
WITH dept_avg AS (
  SELECT Department, AVG(Salary) AS avg_pay
  FROM employees GROUP BY Department)
SELECT e.Full_Name, e.Salary, ROUND(d.avg_pay, 2) AS dept_avg
FROM employees e
JOIN dept_avg d ON e.Department = d.Department
WHERE e.Salary > d.avg_pay
ORDER BY e.Full_Name;
```

**SAMPLE OUTPUT**

| Full_Name       | Salary    | dept_avg |
|-----------------|-----------|----------|
| Alvin Bautista  | 135500.00 | 88986.11 |
| Alvin Luna      | 100000.00 | 98419.64 |
| Alvin Ramos     | 102000.00 | 98419.64 |
| Amalia Alonzo   | 106500.00 | 98419.64 |
| Amalia Bautista | 110000.00 | 88986.11 |
| Andrea Aglipay  | 88000.00  | 76980.77 |
| ...             |           |          |

**10.5** # Top row from every group

```
WITH ranked AS (
  SELECT Full_Name, Department, Salary,
  ROW_NUMBER() OVER (PARTITION BY Department
    ORDER BY Salary DESC,
    Employee_ID) AS rn
  FROM employees WHERE Salary IS NOT NULL)
SELECT * FROM ranked WHERE rn = 1
ORDER BY Department;
```

**SAMPLE OUTPUT**

| Full_Name         | Department      | Salary    | rn |
|-------------------|-----------------|-----------|----|
| Wilfredo Mendoza  | Engineering     | 184000.00 | 1  |
| Alvin Bautista    | Finance         | 135500.00 | 1  |
| Nestor Bonifacio  | Human Resources | 132000.00 | 1  |
| Fernando Castillo | IT              | 160000.00 | 1  |
| Bea Valdez        | Marketing       | 129000.00 | 1  |
| Dennis Guzman     | Operations      | 109500.00 | 1  |
| ...               |                 |           |    |

**10.6** # Query the result of a query

```
SELECT Department, AVG(Salary) AS avg_pay
FROM (
  SELECT * FROM employees WHERE Employment_Type = 'Full-Time'
) AS ft
GROUP BY Department;
```

**SAMPLE OUTPUT**

| Department      | avg_pay      |
|-----------------|--------------|
| Engineering     | 99024.390244 |
| Finance         | 90380.000000 |
| Human Resources | 74934.782609 |
| IT              | 81900.000000 |
| Marketing       | 71250.000000 |
| Operations      | 67500.000000 |

# 11. Window Functions

7 functions

Rank rows without collapsing them.

```
11.1 # Numbers the rows in order
SELECT Full_Name, Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC,
                          Employee_ID) AS position
FROM employees;
```

## SAMPLE OUTPUT

| Full_Name       | Salary    | position |
|-----------------|-----------|----------|
| Alvin Aglipay   | 69500.00  | 159      |
| Alvin Alcantara | 91500.00  | 70       |
| Alvin Bautista  | 135500.00 | 7        |
| Alvin Luna      | 100000.00 | 56       |
| Alvin Ocampo    | NULL      | 242      |
| Alvin Ramos     | 102000.00 | 53       |

```
11.2 # Ranks within each group
SELECT Full_Name, Department, Salary,
       RANK() OVER (PARTITION BY Department
                   ORDER BY Salary DESC) AS dept_rank
FROM employees;
```

## SAMPLE OUTPUT

| Full_Name       | Department  | Salary    | dept_rank |
|-----------------|-------------|-----------|-----------|
| Alvin Aglipay   | Engineering | 69500.00  | 47        |
| Alvin Alcantara | Engineering | 91500.00  | 31        |
| Alvin Bautista  | Finance     | 135500.00 | 1         |
| Alvin Luna      | Engineering | 100000.00 | 25        |
| Alvin Ocampo    | Marketing   | NULL      | 23        |
| Alvin Ramos     | Engineering | 102000.00 | 24        |

```
11.3 # Ranking with no gaps
SELECT Full_Name, Salary,
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
FROM employees;
```

## SAMPLE OUTPUT

| Full_Name       | Salary    | rnk |
|-----------------|-----------|-----|
| Alvin Aglipay   | 69500.00  | 91  |
| Alvin Alcantara | 91500.00  | 53  |
| Alvin Bautista  | 135500.00 | 7   |
| Alvin Luna      | 100000.00 | 43  |
| Alvin Ocampo    | NULL      | 135 |
| Alvin Ramos     | 102000.00 | 40  |

```
11.4 # Group average on every row
SELECT Department, Full_Name, Salary,
       ROUND(AVG(Salary) OVER (PARTITION BY Department), 2) AS dept_avg
FROM employees
ORDER BY Department, Full_Name;
```

## SAMPLE OUTPUT

| Department  | Full_Name        | Salary    | dept_avg |
|-------------|------------------|-----------|----------|
| Engineering | Alvin Aglipay    | 69500.00  | 98419.64 |
| Engineering | Alvin Alcantara  | 91500.00  | 98419.64 |
| Engineering | Alvin Luna       | 100000.00 | 98419.64 |
| Engineering | Alvin Ramos      | 102000.00 | 98419.64 |
| Engineering | Amalia Alonzo    | 106500.00 | 98419.64 |
| Engineering | Amalia Fernandez | 62500.00  | 98419.64 |

**11.5** # Looks at the previous row  
 SELECT Full\_Name, Hired\_Date, Salary,  
       LAG(Salary) OVER (ORDER BY Hired\_Date) AS previous\_hire\_pay  
 FROM employees;

**SAMPLE OUTPUT**

| Full_Name       | Hired_Date | Salary    | previous_hire_pay |
|-----------------|------------|-----------|-------------------|
| Alvin Aglipay   | 2022-12-04 | 69500.00  | 83000.00          |
| Alvin Alcantara | 2022-01-31 | 91500.00  | 79500.00          |
| Alvin Bautista  | 2018-09-25 | 135500.00 | 89000.00          |
| Alvin Luna      | 2026-03-05 | 100000.00 | 33000.00          |
| Alvin Ocampo    | 2016-08-12 | NULL      | 111500.00         |
| Alvin Ramos     | 2022-03-30 | 102000.00 | 71500.00          |

**11.6** # Splits rows into equal buckets  
 SELECT Full\_Name, Salary,  
       NTILE(4) OVER (ORDER BY Salary, Employee\_ID) AS quartile  
 FROM employees  
 WHERE Salary IS NOT NULL  
 ORDER BY Salary;

**SAMPLE OUTPUT**

| Full_Name       | Salary   | quartile |
|-----------------|----------|----------|
| Bea Cruz        | 33000.00 | 1        |
| Bea Quijano     | 34000.00 | 1        |
| Divina Espiritu | 34000.00 | 1        |
| Nestor Santos   | 37000.00 | 1        |
| Zenaida Valdez  | 38000.00 | 1        |
| Josefina Guzman | 38500.00 | 1        |

**11.7** # Running total inside a group  
 SELECT Department, Full\_Name, Salary,  
       SUM(Salary) OVER (PARTITION BY Department  
                           ORDER BY Salary, Employee\_ID) AS running\_total  
 FROM employees  
 WHERE Salary IS NOT NULL  
 ORDER BY Department, Salary;

**SAMPLE OUTPUT**

| Department  | Full_Name          | Salary   | running_total |
|-------------|--------------------|----------|---------------|
| Engineering | Ramon Navarro      | 61500.00 | 61500.00      |
| Engineering | Amalia Fernandez   | 62500.00 | 124000.00     |
| Engineering | Jasmine Cabrera    | 64500.00 | 188500.00     |
| Engineering | Oscar Gatchalian   | 65500.00 | 254000.00     |
| Engineering | Lorenzo Magsaysay  | 66000.00 | 320000.00     |
| Engineering | Veronica Bonifacio | 66000.00 | 386000.00     |

## 12. Insert, Update & Delete

6 functions

Change the data in a table.

**12.1** # Adds one new row  
 INSERT INTO departments (Department, Head, Floor)  
 VALUES ('Legal', 'Nora Aunor', 8);

### SAMPLE OUTPUT

```
+-----+-----+-----+
| Department | Head | Floor |
+-----+-----+-----+
| Engineering | Ramon Villanueva | 5 |
| Finance | Teresita Ocampo | 3 |
| Human Resources | Grace Bautista | 2 |
| IT | Nestor Fernandez | 4 |
| Legal | Nora Aunor | 8 |
| Logistics | Unassigned | 7 |
| ... |  |  |
+-----+-----+-----+
```

**12.2** # Adds several rows at once  
 INSERT INTO departments (Department, Head, Floor) VALUES  
 ('Design', 'Ella Cruz', 9),  
 ('Support', 'Ben Chan', 10);

### SAMPLE OUTPUT

```
+-----+
| total |
+-----+
| 9 |
+-----+
```

**12.3** # Changes values in matching rows  
 UPDATE departments SET Floor = 12  
 WHERE Department = 'Legal';

### SAMPLE OUTPUT

```
department | head | floor
-----+-----+-----
(0 rows)
```

**12.4** # Removes matching rows  
 DELETE FROM departments  
 WHERE Department = 'Support';

### SAMPLE OUTPUT

```
+-----+
| remaining |
+-----+
| 7 |
+-----+
```

### 12.5 Adds a row, or updates it if it exists

#### MySQL

verified on live server

```
INSERT INTO departments (Department, Head, Floor)
VALUES ('Legal', 'New Head', 2)
ON DUPLICATE KEY UPDATE Head = VALUES(Head);
```

#### PostgreSQL

verified on live server

```
INSERT INTO departments (Department, Head, Floor)
VALUES ('Legal', 'New Head', 2)
ON CONFLICT (Department) DO UPDATE SET Head = EXCLUDED.Head;
```

#### SQL Server (T-SQL)

reference only

```
MERGE departments AS t
USING (SELECT 'Legal' AS Department, 'New Head' AS Head) AS s
ON t.Department = s.Department
WHEN MATCHED THEN UPDATE SET Head = s.Head
WHEN NOT MATCHED THEN INSERT (Department, Head) VALUES (s.Department, s.Head);
```

#### MySQL result

```
+-----+-----+-----+
| Department | Head | Floor |
+-----+-----+-----+
| Legal | New Head | 2 |
+-----+-----+-----+
```

#### PostgreSQL result

```
department | head | floor
-----+-----+-----
Legal | New Head | 2
(1 row)
```

**Why they differ:** Upsert is completely different in all three engines - the classic portability trap.

**12.6** # Deletes rows matching a rule  
DELETE FROM departments  
WHERE Floor > 8;

**SAMPLE OUTPUT**

```
+-----+  
| remaining |  
+-----+  
|          7 |  
+-----+
```

# 13. Creating & Altering Tables

7 functions

Build the structure that holds data.

## 13.1 Creates a table with an auto ID

MySQL

verified on live server

```
CREATE TABLE training (
  id INT AUTO_INCREMENT PRIMARY KEY,
  Employee_ID VARCHAR(10),
  Course VARCHAR(60),
  Completed DATE);
```

PostgreSQL

verified on live server

```
CREATE TABLE training (
  id SERIAL PRIMARY KEY,
  Employee_ID VARCHAR(10),
  Course VARCHAR(60),
  Completed DATE);
```

SQL Server (T-SQL)

reference only

```
CREATE TABLE training (
  id INT IDENTITY(1,1) PRIMARY KEY,
  Employee_ID VARCHAR(10),
  Course VARCHAR(60),
  Completed DATE);
```

MySQL result

(statement ran successfully)

PostgreSQL result

(statement ran successfully)

**Why they differ:** AUTO\_INCREMENT vs SERIAL vs IDENTITY(1,1) - one of the first things you hit.

## 13.2 ALTER TABLE training ADD Score INT; # Adds a new column

SAMPLE OUTPUT

(statement ran successfully)

## 13.3 Removes a column

MySQL

verified on live server

```
ALTER TABLE training DROP COLUMN Score;
```

PostgreSQL

verified on live server

```
ALTER TABLE training DROP COLUMN Score;
```

SQL Server (T-SQL)

reference only

```
ALTER TABLE training DROP COLUMN Score;
```

MySQL result

(statement ran successfully)

PostgreSQL result

(statement ran successfully)

**Why they differ:** Same everywhere, but SQL Server needs the constraint dropped first if one exists.

## 13.4 Changes a column type

MySQL

verified on live server

```
ALTER TABLE training MODIFY Course VARCHAR(120);
```

PostgreSQL

verified on live server

```
ALTER TABLE training ALTER COLUMN Course TYPE VARCHAR(120);
```

SQL Server (T-SQL)

reference only

```
ALTER TABLE training ALTER COLUMN Course VARCHAR(120);
```

MySQL result

(statement ran successfully)

PostgreSQL result

(statement ran successfully)

**Why they differ:** MODIFY vs ALTER COLUMN ... TYPE vs ALTER COLUMN - all three differ.

## 13.5 CREATE INDEX idx\_dept ON employees (Department); # Speeds up searches on a column

SAMPLE OUTPUT

(statement ran successfully)

## 13.6 DROP TABLE IF EXISTS training; # Deletes a table completely

SAMPLE OUTPUT

(statement ran successfully)

13.7 Copies a table structure and data

|                                                                                                                                                            |                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>MySQL</div> <div>verified on live server</div> <div>CREATE TABLE eng_staff AS<br/>SELECT * FROM employees<br/>WHERE Department = 'Engineering';</div> | <div>PostgreSQL</div> <div>verified on live server</div> <div>CREATE TABLE eng_staff AS<br/>SELECT * FROM employees<br/>WHERE Department = 'Engineering';</div> |
| <div>SQL Server (T-SQL)</div> <div>reference only</div> <div>SELECT * INTO eng_staff FROM employees<br/>WHERE Department = 'Engineering';</div>            |                                                                                                                                                                 |
| <div>MySQL result</div> <div>+-----+<br/>  copied  <br/>+-----+<br/>       58  <br/>+-----+</div>                                                          | <div>PostgreSQL result</div> <div>copied<br/>-----<br/>58<br/>(1 row)</div>                                                                                     |

**Why they differ:** SQL Server uses SELECT ... INTO; the others use CREATE TABLE AS SELECT.



## 14. Views & Handy Extras

6 functions

Save queries and useful odds and ends.

**14.1** # Saves a query under a name  
 CREATE VIEW managers AS  
 SELECT Full\_Name, Department, Salary  
 FROM employees WHERE Position LIKE '%Manager%';

### SAMPLE OUTPUT

| Full_Name         | Department  | Salary    |
|-------------------|-------------|-----------|
| Wilfredo Mendoza  | Engineering | 184000.00 |
| Fernando Castillo | IT          | 160000.00 |
| Teresita Espiritu | Engineering | 159000.00 |
| Divina Soriano    | Engineering | 150000.00 |
| Katrina Salvador  | Sales       | 148000.00 |
| Oscar Cruz        | IT          | 142000.00 |
| ...               |             |           |

**14.2** DROP VIEW IF EXISTS managers; # Removes a saved query

### SAMPLE OUTPUT

(statement ran successfully)

### 14.3 Shows the tables in the database

**MySQL** verified on live server  
 SHOW TABLES;

**PostgreSQL** verified on live server  
 SELECT table\_name FROM information\_schema.tables  
 WHERE table\_schema = 'public';

**SQL Server (T-SQL)** reference only  
 SELECT name FROM sys.tables;

#### MySQL result

| Tables_in_hr |
|--------------|
| departments  |
| employees    |

#### PostgreSQL result

| table_name  |
|-------------|
| departments |
| employees   |

(2 rows)

**Why they differ:** SHOW is MySQL-only. information\_schema works on MySQL and Postgres.

### 14.4 Shows the columns of a table

**MySQL** verified on live server  
 DESCRIBE employees;

**PostgreSQL** verified on live server  
 SELECT column\_name, data\_type FROM information\_schema.columns  
 WHERE table\_name = 'employees'  
 ORDER BY ordinal\_position;

**SQL Server (T-SQL)** reference only  
 EXEC sp\_columns 'employees';

#### MySQL result

| Field       | Type        | Null | Key | Default | Extra |
|-------------|-------------|------|-----|---------|-------|
| Employee_ID | varchar(10) | NO   | PRI | NULL    |       |
| Full_Name   | varchar(60) | YES  |     | NULL    |       |
| Gender      | varchar(10) | YES  |     | NULL    |       |
| Department  | varchar(30) | YES  |     | NULL    |       |
| Position    | varchar(40) | YES  |     | NULL    |       |
| Hired_Date  | date        | YES  |     | NULL    |       |
| ...         |             |      |     |         |       |

#### PostgreSQL result

| column_name | data_type         |
|-------------|-------------------|
| employee_id | character varying |
| full_name   | character varying |
| gender      | character varying |
| department  | character varying |
| position    | character varying |
| hired_date  | date              |

(6 rows)

**Why they differ:** DESCRIBE is MySQL-only; Postgres uses \d in psql or information\_schema.

**14.5** # A useful summary to keep handy

```
SELECT Department, COUNT(*) AS staff
FROM employees
GROUP BY Department
ORDER BY staff DESC;
```

**SAMPLE OUTPUT**

```
+-----+-----+
| Department | staff |
+-----+-----+
| Engineering | 58 |
| IT          | 40 |
| Finance     | 38 |
| Operations  | 31 |
| Sales       | 31 |
| Human Resources | 26 |
+-----+-----+
```

**14.6 Shows how the database will run a query****MySQL**

verified on live server

```
EXPLAIN SELECT * FROM employees
WHERE Department = 'Engineering';
```

**PostgreSQL**

verified on live server

```
EXPLAIN SELECT * FROM employees
WHERE Department = 'Engineering';
```

**SQL Server (T-SQL)**

reference only

```
SET SHOWPLAN_ALL ON; -- then run the query
```

**MySQL result**

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Ex...
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | employees | ALL | NULL | NULL | NULL | NULL | 246 | Us...
```

**PostgreSQL result**

```
QUERY PLAN
-----
Seq Scan on employees (cost=0.00..11.00 rows=1 width=956)
  Filter: ((department)::text = 'Engineering'::text)
(2 rows)
```

**Why they differ:** Use this to check whether your index is actually being used.

# 15. Dialect Comparison Tables

38 functions

Every difference in this guide, side by side.

## Dates & Times

| What you want     | MySQL                                     | PostgreSQL                                | SQL Server                                       |
|-------------------|-------------------------------------------|-------------------------------------------|--------------------------------------------------|
| Today's date      | <code>CURDATE()</code>                    | <code>CURRENT_DATE</code>                 | <code>CAST(GETDATE() AS DATE)</code>             |
| Current timestamp | <code>NOW()</code>                        | <code>NOW()</code>                        | <code>GETDATE()</code>                           |
| Text -> date      | <code>STR_TO_DATE(s, '%d-%b-%Y')</code>   | <code>TO_DATE(s, 'DD-Mon-YYYY')</code>    | <code>CONVERT(DATE, s, 106)</code>               |
| Date -> text      | <code>DATE_FORMAT(d, '%d/%m/%Y')</code>   | <code>TO_CHAR(d, 'DD/MM/YYYY')</code>     | <code>FORMAT(d, 'dd/MM/yyyy')</code>             |
| Add 30 days       | <code>DATE_ADD(d, INTERVAL 30 DAY)</code> | <code>d + INTERVAL '30 days'</code>       | <code>DATEADD(DAY, 30, d)</code>                 |
| Days between      | <code>DATEDIFF(a, b)</code>               | <code>a - b</code>                        | <code>DATEDIFF(DAY, b, a)</code>                 |
| Years between     | <code>TIMESTAMPDIFF(YEAR, a, b)</code>    | <code>EXTRACT(YEAR FROM AGE(b, a))</code> | <code>DATEDIFF(YEAR, a, b)</code>                |
| Extract year      | <code>YEAR(d)</code>                      | <code>EXTRACT(YEAR FROM d)</code>         | <code>YEAR(d)</code>                             |
| Start of month    | <code>DATE_FORMAT(d, '%Y-%m-01')</code>   | <code>DATE_TRUNC('month', d)</code>       | <code>DATEFROMPARTS(YEAR(d), MONTH(d), 1)</code> |
| Date literal      | <code>'2022-12-04'</code>                 | <code>DATE '2022-12-04'</code>            | <code>'20221204'</code>                          |

## Text & Strings

| What you want         | MySQL                           | PostgreSQL                      | SQL Server                      |
|-----------------------|---------------------------------|---------------------------------|---------------------------------|
| Join text             | <code>CONCAT(a, b)</code>       | <code>a    b</code>             | <code>a + b</code>              |
| Length                | <code>CHAR_LENGTH(s)</code>     | <code>LENGTH(s)</code>          | <code>LEN(s)</code>             |
| Position of text      | <code>LOCATE('@', s)</code>     | <code>POSITION('@' IN s)</code> | <code>CHARINDEX('@', s)</code>  |
| Substring             | <code>SUBSTRING(s, 4, 4)</code> | <code>SUBSTRING(s, 4, 4)</code> | <code>SUBSTRING(s, 4, 4)</code> |
| Case-insensitive LIKE | <code>LIKE (default)</code>     | <code>ILIKE</code>              | <code>LIKE (default)</code>     |
| NULL fallback         | <code>IFNULL(a, b)</code>       | <code>COALESCE(a, b)</code>     | <code>ISNULL(a, b)</code>       |
| Quote identifier      | <code>`column`</code>           | <code>"column"</code>           | <code>[column]</code>           |

## Paging, Limits & NULLs

| What you want           | MySQL                              | PostgreSQL                         | SQL Server                                         |
|-------------------------|------------------------------------|------------------------------------|----------------------------------------------------|
| First 5 rows            | <code>LIMIT 5</code>               | <code>LIMIT 5</code>               | <code>SELECT TOP 5</code>                          |
| Skip 10, take 5         | <code>LIMIT 5 OFFSET 10</code>     | <code>LIMIT 5 OFFSET 10</code>     | <code>OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY</code> |
| NULLs when sorting ASC  | <code>NULLs first</code>           | <code>NULLs last</code>            | <code>NULLs first</code>                           |
| NULLs when sorting DESC | <code>NULLs last</code>            | <code>NULLs first</code>           | <code>NULLs last</code>                            |
| Force NULLs last        | <code>ORDER BY x IS NULL, x</code> | <code>ORDER BY x NULLS LAST</code> | <code>ORDER BY CASE WHEN x IS NULL...</code>       |
| Unquoted name case      | <code>kept as written</code>       | <code>folded to lowercase</code>   | <code>kept as written</code>                       |

## Tables, Keys & Upsert

| What you want      | MySQL                    | PostgreSQL                  | SQL Server                     |
|--------------------|--------------------------|-----------------------------|--------------------------------|
| Auto-increment key | INT AUTO_INCREMENT       | SERIAL / GENERATED IDENTITY | INT IDENTITY(1,1)              |
| Insert or update   | ON DUPLICATE KEY UPDATE  | ON CONFLICT ... DO UPDATE   | MERGE                          |
| Change column type | MODIFY col TYPE          | ALTER COLUMN col TYPE t     | ALTER COLUMN col t             |
| Copy table         | CREATE TABLE t AS SELECT | CREATE TABLE t AS SELECT    | SELECT ... INTO t              |
| Drop if present    | DROP TABLE IF EXISTS t   | DROP TABLE IF EXISTS t      | DROP TABLE IF EXISTS t (2016+) |
| List tables        | SHOW TABLES              | information_schema.tables   | sys.tables                     |
| Describe a table   | DESCRIBE t               | \d t (in psql)              | EXEC sp_columns 't'            |

## Types & Odds and Ends

| What you want           | MySQL                     | PostgreSQL                    | SQL Server               |
|-------------------------|---------------------------|-------------------------------|--------------------------|
| Text type               | VARCHAR / TEXT            | VARCHAR / TEXT                | VARCHAR / NVARCHAR       |
| Boolean                 | TINYINT(1)                | BOOLEAN                       | BIT                      |
| Money-safe number       | DECIMAL(10,2)             | NUMERIC(10,2)                 | DECIMAL(10,2)            |
| Cast a value            | CAST(x AS DECIMAL(10,2))  | x::NUMERIC(10,2)              | CAST(x AS DECIMAL(10,2)) |
| Integer division        | 5 DIV 2 -> 2              | 5 / 2 -> 2                    | 5 / 2 -> 2               |
| String concat with NULL | CONCAT ignores NULL       | gives NULL                    | + gives NULL             |
| No FULL OUTER JOIN      | not supported (use UNION) | supported                     | supported                |
| IF shorthand            | IF(c, a, b)               | CASE WHEN c THEN a ELSE b END | IIF(c, a, b)             |